

Clearance — POC Specification

Clearance — POC Specification

What It Is

Clearance is an MCP server that replaces diff-browsing with a conversational review experience. You chat with an agent that has already read the diff; it briefs you, answers your questions, and surfaces relevant code inline. You direct; it writes the comments.

It works with GitHub PRs and local git diffs using the same pipeline.

V1 Scope

Feature	Status
GitHub PR checkout via OAuth + bare clone + worktree	v1
Local diff support (<code>git diff <base>..<code><head></code></code>)	v1
Diff parsing and structured hunk model	v1
Opening briefing from the agent	v1
Conversational Q&A with inline diff surfacing in MCP App UI	v1
Staged comments panel + GitHub submission	v1
Persistent review instructions (global + per-repo)	v1
Review chapters (large PR decomposition)	post-MVP
Surprise detector (anomaly flagging)	post-MVP
“Brief me in 2 minutes” mode	post-MVP
Learns your patterns over time	post-MVP

Installation

Distributed via npm, invoked via npx:

```
npx clearance auth           # one-time GitHub OAuth device flow
npx clearance review --pr 123 --repo owner/repo
npx clearance review         # defaults to main..HEAD in cwd
```

Added to MCP config as:

```
{
  "mcpServers": {
    "clearance": {
      "command": "npx",
      "args": ["clearance"]
    }
  }
}
```

Authentication

GitHub auth uses the **OAuth device flow** — `clearance auth` opens a browser prompt, the user approves, and the resulting token is stored in the **system keychain** via `keytar`. Never written to disk or environment variables. Subsequent server starts pull the token from the keychain silently.

Diff Source Abstraction

The core pipeline operates on a `DiffSource` interface. Two implementations in v1:

```
interface DiffSource {
  getMetadata(): Promise<DiffMetadata> // title, description,
    author, linked issue
  getDiff(): Promise<ParsedDiff> // structured hunk model
  getFileContent(path: string, ref: string): Promise<string>
}
```

```
class GitHubPRDiffSource implements DiffSource { ... }
class LocalGitDiffSource implements DiffSource { ... }
```

`LocalGitDiffSource` accepts two git refs (base, head) and defaults to `main..HEAD`. This means any two refs work: `main..HEAD`, `HEAD~1..HEAD`, `v1.2.0..v1.3.0`, a specific commit SHA, etc.

This abstraction ensures no GitHub-specific assumptions leak into the analysis or UI layers.

Repo Management

- One **bare clone** per repo stored in `~/.clearance/repos/<owner>/<repo>.git`

- Each PR gets its own **git worktree** (`git worktree add`) branched off the fetched PR ref
 - Worktrees are created on session start, removed on session end (`git worktree remove`)
 - The bare clone is shallow (`--depth=1`) and updated via `git fetch` on demand
 - Local diff reviews operate directly on the current working directory — no clone needed
-

Diff Parsing

Raw git diff is parsed into a structured model using **parse-diff**:

```
interface ParsedDiff {
  files: ParsedFile[]
}

interface ParsedFile {
  path: string
  hunks: Hunk[]
}

interface Hunk {
  oldStart: number
  newStart: number
  lines: DiffLine[]
}

interface DiffLine {
  type: 'add' | 'del' | 'context'
  oldNumber?: number
  newNumber?: number
  content: string
}
```

This model is what the agent reasons over. All line number references in conversation map back to this structure.

MCP Tool Surface

Discrete tools — the agent orchestrates them, the conversational feel comes from the agent, not from monolithic tools:

Tool	Description
start_review	Initialise a review session, checkout diff source, return metadata + opening briefing
get_diff_summary	Return structured summary of all changed files and hunk counts

Tool	Description
get_file_hunk	Return lines N–M of a specific file from the diff
get_file_content	Return full file content at a given ref
search_diff	Find diff lines matching a pattern
stage_comment	Queue a comment at a file + line
list_staged_comments	Return all pending comments
edit_staged_comment	Update a queued comment
discard_staged_comment	Remove a queued comment
submit_review	Post all staged comments to GitHub as a review submission

MCP App UI

Built with **React + Vite**, rendered as a sandboxed iframe via the MCP Apps spec (text/html;profile=mcp-app). Communicates with the MCP server bidirectionally via postMessage JSON-RPC.

UI panels

Diff viewer — react-diff-viewer-continued. Unified view by default, toggle to split. Displays selective hunks as surfaced by the agent (not the full diff). Line numbers preserved from the original diff model. Clicking a line inserts a reference (e.g. index.ts:42) into the chat input.

Staged comments panel — lists all pending comments with file, line, and body. Each comment is editable and discardable inline. “Submit Review” button posts to GitHub.

Session header — PR title (or ref range for local diffs), author, branch, linked issue if present.

Review Instructions

Plain markdown files, loaded at session start and injected into the agent’s context:

Path	Scope
~/.clearance/instructions.md	Global — applies to all reviews
.clearance/instructions.md	Per-repo — committed to the repo root, benefits the whole team

clearance config opens the appropriate file in \$EDITOR as a convenience. The files are the source of truth.

Opening Briefing

On `start_review`, the agent is given: - The full `ParsedDiff` summary (files changed, hunk counts, line counts) - PR metadata or ref range description - The linked issue body (if GitHub PR) - Global + project review instructions

It produces a structured briefing: 1. What this change is actually doing in one paragraph 2. The 2–3 things the reviewer is most likely to care about (informed by instructions) 3. Anything that surprised it (pattern breaks, unexpected file touches, large size vs. simple description)

Stack Summary

Concern	Choice
Language	TypeScript
Distribution	npx
MCP SDK	@modelcontextprotocol/sdk
GitHub API	Octokit (@octokit/rest)
Auth storage	keytar (system keychain)
Diff parsing	parse-diff
UI framework	React + Vite
Diff viewer component	react-diff-viewer-continued
Repo/worktree management	simple-git
MCP App rendering	MCP Apps spec (SEP-1865)

Open Questions

- Should `.clearance/instructions.md` be gitignored by default (personal) or committed (team)? Probably user’s choice, but the default should be committed.
- How does the MCP App UI handle hosts that don’t support MCP Apps yet (e.g. fallback to text-only diff references)?
- Rate limiting strategy for GitHub API calls during analysis of large PRs.
- Should `submit_review` support “Request Changes” and “Approve” review states, or just “Comment” for v1?